

Módulo 9: Evaluación

Text Mining en *Python*

Carga de datos

Desde la sesión de R, cargamos el archivo *evaluacion.RData*, que contiene tres datasets: *prado*, *reviews*.y *documentos*. Exportamos dichos datasets como CSV.

```
# Desde la sesión de R
load("evaluacion.RData")

readr::write_delim(prado, "prado.csv", delim = "\t")
readr::write_delim(reviews, "reviews.csv", delim = "\t")
readr::write_delim(documentos, "documentos.csv", delim = "\t")
```

Cargamos los datos.

```
from pandas import read_csv

prado = read_csv("prado.csv", sep = "\t", dtype = {"status":str})
reviews = read_csv("reviews.csv", sep = "\t")
documentos = read_csv("documentos.csv", sep = "\t")
```

Descripción de los datasets:

- **prado**: Cuenta de X (antiguo Twitter) del *Museo del Prado*: 1.038 tuits (desde 01-04-2025 al 31-07-2026).
- **reviews**: Contiene 13.000 críticas sobre el alojamiento en diversas ciudades de Andalucía.
- **documentos**: Contiene 120 oraciones.

Pregunta 1. Text Mining

El dataset **prado** contiene 1.038 tuits, de la cuenta del *Museo del Prado*. Descripción de las columnas:

- **status**: Identificador del tuit
- **fecha**: Fecha de publicación del tuit. Desde el 01-04-2025 hasta el 31-07-2026.
- **texto**: El texto del tuit.

Para realizar *text mining* al texto de los tuits, es importante **depurar previamente el texto**.

Como, por ejemplo, eliminar del texto: usuarios de X, hashtags, URLs, ...

Para realizar la *limpieza del texto*, podemos utilizar el siguiente script:

```
# === Limpieza de Texto ===
import re, string

prado['textmining'] = prado['texto']
# Quitar "@usuario"
prado['textmining'] = prado['textmining'].str.replace(r'@\w+', '', regex=True)
# Quitar "#hashtag"
prado['textmining'] = prado['textmining'].str.replace(r'#\w+', '', regex=True)
# Quitar "URL"
prado['textmining'] = prado['textmining'].str.replace(r'http\S*', '', regex=True)
# Quitar "e-Mail"
prado['textmining'] = prado['textmining'].str.replace(r'\w+@\w+\.\w+', '', regex=True)
# Si fuera necesario, añadir espacio después de "punto", "coma", "punto y coma", "dos puntos"
prado['textmining'] = prado['textmining'].str.replace(
    r'([.,;:])([a-zA-ZáéíóúñÁÉÍÓÚÑ])', r'\1 \2', regex=True
)
# Conservar letras, dígitos, espacios en blanco, saltos de línea y caracteres de puntuación
prado['textmining'] = prado['textmining'].str.replace(
    r'[^\wÀ-ÿ' + re.escape(string.punctuation) + r'\s]', '', regex=True
)
```

```
# Quitar símbolos
prado['textmining'] = prado['textmining'].str.replace('!', '')
# Quitar espacios en blanco al inicio y final del texto
prado['textmining'] = prado['textmining'].str.strip()
```

Realizar *text mining* a los tuits de la cuenta *prado*, desde 01-06-2025 hasta 31-07-2026.

Utilizar las librerías *spacy* y *nltk*, según proceda.

Trabajar con la columna "*textmining*". Operar con sustantivos, adjetivos y nombres propios.

- Términos más frecuentes.
- Buscar *palabras clave*:
 - N-gramas: obtener bigramas, trigramas, ...
 - Colocaciones: obtener bigramas, trigramas, ...
- Visualizar las *coocurrencias*.
- Reconocimiento de entidades nombradas (*NER*).
- Podemos clasificar los tuits conforme a una temática determinada. Para extraer temas, podemos basarnos en la información que se desprende del *text mining*. Se pide:
 - Examinando las palabras clave (bigramas, trigramas, etc) y las coocurrencias obtenidas, indica algunos nombres de *pintores y exposiciones*.
Si te resulta más cómodo, señala las relaciones oportunas (término1 – término2 – término3 – término4).
 - Selecciona aquellos tuits donde figure la cadena de texto "*Veronese*". Después, muestra el texto de los cinco primeros tuits (ordenados por fecha, del tuit más antiguo al más reciente). Para finalizar, visualiza las *coocurrencias*.
 - Ídem a la pregunta anterior, pero empleando la *cadena de texto* "*femenino*".
- *Word embeddings*:
 - Modelo *Word2Vec*. El texto debe estar compuesto únicamente por los lemas deseados.
 - Mostrar las *similitudes* de los términos: "*farnesio*", "*caravaggio*", "*università*", "*falomir*".
 - Visualizar word embeddings mediante el método *UMAP*.

Pregunta 2. Clasificación de Texto: Análisis de sentimiento

El dataset *reviews* contiene 13.000 críticas sobre el alojamiento en diversas ciudades de Andalucía. Descripción de las columnas:

- *titulo*: Breve descripción de la crítica.
- *texto*: El texto de la crítica.
- *puntuacion*: La calificación del usuario, en una escala de 5 estrellas. Variable categórica, con cinco valores posibles: 1 (muy mal), 2 (mal), 3 (regular), 4 (bien), 5 (muy bien).

Se pide:

- Divida *reviews* en dos datasets: dataset de *entrenamiento* (con el 80% de las filas) y dataset de *prueba* (con el 20% de las filas restantes). No es necesario que los datasets estén balanceados.
- Mediante la herramienta *fastText*, cree un *modelo de clasificación de texto* basándose en la variable *puntuacion*.
- En *reviews*, cree una variable categórica, denominada *sentimiento*, siguiendo estas indicaciones:
 - Si *puntuacion* es menor de 3, tomará el valor *negativo*.
 - Si *puntuacion* es igual a 3, tomará el valor *neutro*.
 - Si *puntuacion* es mayor de 3, tomará el valor *positivo*.

Empleando la herramienta *fastText*, cree un *modelo de clasificación de texto* basándose en la variable *sentimiento*.

- Utilizando la herramienta *fastText*, cree un *modelo de clasificación de texto* basándose en la variable *sentimiento*, pero seleccionando únicamente los valores negativo o positivo, es decir, prescindiendo de aquellas filas donde el sentimiento sea neutro.

- Empleando la **Regresión Logística**, cree un *modelo de clasificación de texto* basándose en la variable **sentimiento**, pero seleccionando únicamente los valores **negativo** o **positivo**.
- Usando el algoritmo **Naive Bayes**, cree un *modelo de clasificación de texto* basándose en la variable **sentimiento**, pero seleccionando únicamente los valores **negativo** o **positivo**.

Pregunta 3. Grandes Modelos de Lenguaje (LLM)

El dataset **documentos** contiene 120 oraciones. Descripción de las columnas:

- *doc_id*: Identificador del documento.
- *texto*: El texto del documento.

Sabemos de antemano que cada uno de los documentos pertenece a una de estas ocho **categorías**: *Cine, Cocina, Deportes, Economía, Historia, Literatura, Naturaleza, Tecnología*. De cada categoría hay 15 documentos.

Se pide:

- **Clustering con BERT**

Empleando *Sentence Transformers*, utilizar el modelo `hiiamsid/sentence_similarity_spanish_es` para obtener los *embeddings* de los documentos.

Visualizar los embeddings mediante *UMAP*.

Realizar el *clustering*, asignando a cada documento una de las ocho categorías anteriormente indicadas.

- **Clustering con Nomic Bert**

Empleando *Ollama*, usar el modelo `nomic-embed-text-v2-moe` para obtener los *embeddings* de los documentos.

Visualizar los embeddings mediante *UMAP*.

Realizar el *clustering*, asignando a cada documento una de las ocho categorías anteriormente señaladas.

- **Clasificación de texto con Qwen3**

Empleando *Ollama*, usar el modelo `qwen3:4b` para clasificar los documentos, asignando a cada documento una de las ocho categorías anteriormente mencionadas.

Recomendaciones a seguir: No utilizar la salida estructurada, incluir en el *prompt* las ocho categorías. El *thinking* debe estar activado.

- **Comparar la clasificación de Qwen3 con el clustering de BERT**

Mostrar aquellos documentos no coincidentes, es decir, la categoría asignada con *Qwen3* es distinta al grupo asignado con *BERT*.

- **Comparar la clasificación de Qwen3 con el clustering de Nomic Bert**

Mostrar aquellos documentos no coincidentes, es decir, la categoría asignada con *Qwen3* es distinta al grupo asignado con *Nomic Bert*.

Pregunta 4. Grandes Modelos de Lenguaje (LLM)

Queremos extraer cierta información de varias páginas web de *Wikipedia* (URL), referentes a 10 personajes ilustres: "Albert Camus", "Marie Curie", "Albert Einstein", "Alexander Fleming", "Mahatma Gandhi", "Gabriel García Márquez", "Nelson Mandela", "Severo Ochoa", "Max Planck" y "Santiago Ramón y Cajal".

Páginas web (URL) a procesar:

```
urls = [
    "https://es.wikipedia.org/wiki/Albert_Camus",
    "https://es.wikipedia.org/wiki/Marie_Curie",
    "https://es.wikipedia.org/wiki/Albert_Einstein",
    "https://es.wikipedia.org/wiki/Alexander_Fleming",
    "https://es.wikipedia.org/wiki/Mahatma_Gandhi",
    "https://es.wikipedia.org/wiki/Gabriel_García_Márquez",
    "https://es.wikipedia.org/wiki/Nelson_Mandela",
    "https://es.wikipedia.org/wiki/Severo_Ochoa",
    "https://es.wikipedia.org/wiki/Max_Planck",
    "https://es.wikipedia.org/wiki/Santiago_Ramón_y_Cajal"]
```

De cada personaje, queremos obtener la siguiente información:

- *Nombre*: Nombre de la persona.
- *Nacimiento*: Fecha y lugar de nacimiento.
- *Resumen*: Un resumen de su principal logro.
- *Nobel*: Disciplina en la que ganó el Premio Nobel.
- *Año*: Año en que ganó el Premio Nobel.
- *Fallecimiento*: Fecha y lugar de fallecimiento.

Se puede dar el caso, en el que algún personaje tenga más de un premio nobel (*Marie Curie*).

También, se puede dar el caso, en el que algún personaje nunca fue galardonado con el premio nobel (*Mahatma Gandhi*).

Se pide:

- Utilizar la *API Web fetch* de Ollama para extraer el **contenido** de las páginas web indicadas.

- **Gemini**

Utilizando la librería `chatlas`:

- Definir la salida estructurada a utilizar en el modelo.
- Usar el modelo *Gemini* para obtener la información requerida.

Es aconsejable hacer una pausa (30 o 45 segundos) entre peticiones, para no saturar el servidor.

- **Ollama**

- Definir la salida estructurada a utilizar en el modelo.
- Usar el modelo `qwen3:4b` para obtener la información requerida.